BC-FL-SRV: Presentation & Viva Defense Guide

Verifiable Admission Decisions for Privacy-Preserving Federated Learning

Department: Computer Engineering, SIES GST

Academic Year: 2026–27 (Semester VII Stage-I)

Group Members: Ayush Lanke (123A7004), Bhumi Beloshe (123A7005), Sakshi Bhandare (123A7007)

Implementation Timeline: 1 August 2026 to 5 October 2026

1. THE 30-SECOND ELEVATOR PITCH (THE REAL-WORLD ANALOGY)

The "Blindfolded Teacher" Analogy:
Imagine students taking an exam from home (Federated Learning). To prevent cheating, they put their answers inside sealed, opaque envelopes (Secure Aggregation). The teacher adds up all the test scores while completely blindfolded so they cannot spy on any single student's paper. That protects privacy.

The Big Problem: Because the teacher is blindfolded, *nobody can check whether the teacher secretly threw away honest students' papers, included fake exam sheets from unauthorized students, or applied the declared grading curve honestly!*

Our Solution (BC-FL-SRV): We use an immutable cryptographic audit log (RFC 6962 Merkle Tree) and a rotating committee of elected participants who inspect bounded mathematical sketches of updates, threshold-sign the admission decision, and publish it so **any external auditor can mathematically verify fairness in microseconds without seeing raw private data**.

2. CORE CONCEPTS CHEAT-SHEET (INSTANT PLAIN-ENGLISH DEFINITIONS)

Concept	What It Actually Means (Plain English)
Federated Learning (FL)	Decentralized AI training: edge devices train locally and send only weight deltas to a central server; raw data never leaves the device.
Secure Aggregation (SecAgg)	Pairwise additive masking where masks cancel out when added together. Server sees *only the sum*, not individual contributions.
The Accountability Gap	Because updates are masked, the server can secretly drop honest clients or inject poisoned updates without anyone being able to prove it.
AG1: Correct Admission	Proving that only clients who met the declared admission rule were included in the round.
AG2: Correct Aggregation	Proving the mathematical vector sum was calculated correctly (traditionally requires slow Zero-Knowledge Proofs).
AG3: Non-Equivocation	Proving the server cannot show different models or histories to different participants (forking the state).
RFC 6962 Merkle Tree	The IETF standard used by Google Certificate Transparency. Guarantees tamper-evidence and microsecond inclusion proofs.
Verifiable Sortition	A fair cryptographic lottery that picks an attestation committee using the previous round's Merkle root as a tamper-proof seed.

Bounded Evidence Ladder (E0–E4)	Five privacy tiers: E0 (cryptographic commitment only) to E3 (64-dimensional random projection sketch) to E4 (full plaintext update).
G2 Admission Rule	Our two-factor filter: checks both update magnitude (norm) and low-dimensional cosine direction against a running momentum reference.

3. WORD-FOR-WORD PRESENTATION SCRIPT (5-MINUTE SPEAKING GUIDE)

Step 1: Introduction (30 seconds)

*"Good morning respected guide and faculty members. We are Ayush, Bhumi, and Sakshi. Today we present our Stage-I Major Project titled **BC-FL-SRV: Verifiable Admission Decisions for Privacy-Preserving Federated Learning**. In short, our system solves a major vulnerability in decentralized machine learning: how to guarantee that the central server does not cheat when admitting participants, while preserving complete privacy under secure aggregation."*

Step 2: The Core Problem (45 seconds)

"Federated learning protects data privacy by training models locally on edge devices. To prevent the server from spying on gradients, Secure Aggregation masks individual updates so the server only sees the aggregate sum. However, this creates a critical accountability gap: because updates are hidden, the server has unchecked authority. It can secretly discard honest participants or admit poisoned updates, and no external party can verify it.
Recent literature from PoPETs 2026 shows that over 75% of research focuses solely on verifying aggregation using computationally heavy Zero-Knowledge Proofs. In contrast, verifiable admission (AG1) and non-equivocation (AG3) have been largely overlooked, despite being fundamentally lighter and cheaper to verify."

Step 3: Our Proposed System Architecture (1 minute)

"To solve this without needing slow Zero-Knowledge Proofs or proprietary secure hardware, we developed BC-FL-SRV based on three architectural pillars:
***First**, an append-only RFC 6962 Merkle Transcript that immutably logs every client submission, admission set, and aggregation.*
***Second**, a Rotating Attestation Committee selected via verifiable sortition seeded from the previous Merkle root. Committee members inspect bounded privacy-preserving sketches (E3) and threshold-sign the admission set.*
***Third**, a Stateless External Auditor that can verify AG1 and AG3 from the public Merkle root alone in microseconds."*

Step 4: Experimental Results & Implementation (1 minute)

"Between 1st August and 5th October 2026, we completed the core system implementation and conducted seven benchmark experiments:
- ***Model Size Invariance:** Auditor verification takes just 1.5 microseconds and is strictly independent of model size (a 1.03× ratio across a 65,536× model size scaling range).*
- ***Tamper Detection:** Our protocol detects 12 out of 14 adversarial attack vectors with zero false alarms.*
- ***Machine Learning Utility:** On Fashion-MNIST under Dirichlet non-IID partitioning, our system reaches 83.88% test accuracy with exactly zero accuracy loss compared to an unmasked baseline.*
- ***Code Rigor:** We have 366 automated unit and security tests passing with 99% overall code coverage."*

Step 5: Stage-II Roadmap & Closing (30 seconds)

"For Stage-II next semester, we will conduct component ablation studies, simulate peer-to-peer auditor gossip networks to prevent split-view attacks, and prepare our findings for conference publication. Thank you, and we welcome your questions."

4. TOP 10 QUESTIONS FACULTY WILL ASK & WINNING ANSWERS

Q1: Why did you use an RFC 6962 Merkle tree instead of Ethereum or Hyperledger?

Answer: "Storing frequent model updates on Ethereum is cost-prohibitive in gas fees and leaks privacy on a public chain. RFC 6962 Merkle trees—the exact cryptographic standard powering Google's Certificate Transparency—provide the exact same tamper-evidence, immutability, and O(log n) inclusion proofs at zero cost and microsecond verification speed."

Q2: What is the difference between AG1, AG2, and AG3?

Answer: "These are formal integrity claims from the PoPETs 2026 systematization: **AG1 (Correct Admission)** proves only eligible clients were admitted; **AG2 (Correct Aggregation)** proves the sum was computed correctly; and **AG3 (Non-Equivocation)** proves the server shows the same history and model to everyone."

Q3: How can the committee check updates without seeing raw private client data?

Answer: "We designed a bounded evidence ladder (E0 to E4). Instead of inspecting raw weights, the committee inspects a bounded sketch (E3)—a 64-dimensional random Johnson-Lindenstrauss projection that preserves norm and directional cosine similarity without revealing the raw training dataset."

Q4: What dataset, neural network, and hyperparameters did you use?

Answer: "We evaluated on Fashion-MNIST using a Convolutional Neural Network (SmallCNN) across 5 clients. To simulate real edge heterogeneity, we partitioned data using a non-IID Dirichlet distribution with concentration parameter $\alpha = 0.5$, reaching 83.88% accuracy in 10 rounds."

Q5: What happens if committee members collude or are corrupted?

Answer: "We use threshold multi-signatures (t of m). In Experiment E-3, we analyzed corruption using a hypergeometric probability model. While a single self-attesting aggregator has a 20% forge chance at 20% corruption, an attested committee of size $m = 15$ suppresses the forgery probability to 2.92×10^{-7}."

Q6: What are the two attacks your system could NOT detect? (Honest Negative Results)

Answer: "We detected 12 of 14 attacks. The two undetected cases are: (1) Selective client dropping where only the excluded client observes its absence (resolved via client-side inclusion proofs), and (2) Split-view equivocation when only one auditor is present (which we are addressing in Stage-II via multi-auditor gossip protocols)."

Q7: Why does verification cost NOT increase when the AI model gets bigger?

Answer: "Because the auditor only verifies cryptographic hashes and Ed25519 digital signatures on the Merkle tree nodes. Regardless of whether the model has 1,000 parameters (64 bytes) or 1 million parameters (4 MiB), the SHA-256 hash is always exactly 32 bytes, keeping verification time constant at ~1.5 microseconds."

Q8: How does verifiable sortition prevent the server from rigging the committee?

Answer: "The random seed used to pick committee members is derived deterministically from the Merkle root of the previous finalized round (R_{t-1}). Because the root is committed cryptographically before round t begins, the server cannot manipulate the seed without invalidating past history."

Q9: How do you know the code is bug-free and handles malicious inputs?

Answer: "We wrote 366 automated test cases with 99% line coverage. Our tests specifically assert that every verifier 'fails closed'—meaning malformed, truncated, or tampered bytes immediately reject without crashing the system."

Q10: What is your Stage-II plan for Semester VIII?

Answer: "Stage-II focuses on: (1) Component ablation studies (Experiment E-7) to prove the necessity of each component, (2) Multi-auditor gossip simulations (Experiment E-6), and (3) Compiling our research findings into a conference paper manuscript."

5. LIVE DEMONSTRATION CHEAT-SHEET (COMMANDS TO RUN)

> **Pro-Tip for Tomorrow:** If the faculty asks to see the project running, keep a terminal window open in `/Users/ayush/Downloads/Study`. You have two options that work reliably:

Option 1: Run the 366 Automated Tests (Takes ~5 Seconds)

Shows instant engineering maturity and 100% test pass rate in green text:

```
.venv_mac/bin/python -m pytest tests/ -v
```

Option 2: Launch the Interactive Web Dashboard

Opens a web application at `http://127.0.0.1:8000` showing Merkle blocks, transactions, and attack simulations:

```
.venv_mac/bin/python scripts/run_ui.py
```

Option 3: Run the Decentralized FL Simulation Demo

Runs a live 5-client training round simulation on the console:

```
.venv_mac/bin/python scripts/run_demo.py --rounds 3 --quiet
```

6. REPOSITORY DIRECTORY STRUCTURE REFERENCE

Folder / Module	Responsibility in the Project
`src/bcflsrv/ledger/`	RFC 6962 Merkle tree, inclusion proofs, consistency proofs, typed transcript.
`src/bcflsrv/crypto/`	Ed25519 digital signature keys, verifiable committee sortition engine.
`src/bcflsrv/protocol/`	Admission sets, multi-signature threshold attestation, external auditor engine.
`src/bcflsrv/fl/`	PyTorch SmallCNN, Fashion-MNIST, Dirichlet non-IID partition, E0-E4 ladder, G1/G2 rules.
`src/bcflsrv/bench/`	Microsecond-precision benchmark harnesses and parameter scaling utilities.
`src/bcflsrv/ui/`	FastAPI web backend and interactive HTML5 Canvas research dashboard.
`scripts/`	Benchmark runners for E-1 (scaling), E-2 (attacks), E-3 (corruption), E-4/E-5 (trade-off).
`tests/`	366 comprehensive unit, protocol, security, and UI integration test cases (99% coverage).

7. CODE WALKTHROUGH: EXPLAINING MERKLE.PY (LINES 117–157)

> **Why this matters:** If the faculty asks you to open the codebase and explain how the Merkle tree works, navigate to `src/bcflsrv/ledger/merkle.py` and explain these exact lines:

Line 119: What is self._stack = []?

Explanation: "It stores subtrees as tuples of `(subtree_root, subtree_size)`. The sizes are strictly decreasing from left to right (like powers of 2: 8, 4, 2, 1). This allows us to append new leaves incrementally without rebuilding the entire tree from scratch."

Lines 129–134: How does append() merge equal-sized neighbours?

Explanation: "When a new leaf is added, it pushes `(leaf_hash, 1)` to the stack. The while-loop checks if the top two items have equal sizes. If both subtrees have size 1, they merge into a parent of size 2. If the stack now has two trees of size 2, they merge into size 4. This is identical to **binary carry addition** (1 + 1 = 10 in binary), giving an O(log n) append amortized time."

Lines 137–144: How is the current root computed?

Explanation: "If there are remaining unequal subtrees on the stack (for instance, a tree of size 4 and size 1), the property folds them together from left to right using `hash_children(left_root, right_root)`, producing the exact canonical Merkle Tree Hash specified in RFC 6962."

Lines 147–157: What is inclusion_proof()?

Explanation: "It generates the audit path for a leaf at a given index. It recursively splits the tree at the largest power of two below n (`k = _largest_power_of_two_below(n)`), providing the sequence of sibling hashes needed for an auditor to recompute the root in O(log n) hash steps."

8. LAST-MINUTE 10-HOUR CHECKLIST FOR TOMORROW

- **1. Open Terminal in Project Folder:** Run `cd /Users/ayush/Downloads/Study` and make sure `.venv_mac` is ready.
- **2. Practice Running Pytest Once Tonight:** Run `.venv_mac/bin/python -m pytest tests/ -q` so you see the green dots yourself.
- **3. Read the 5-Minute Script Twice Out Loud:** Don't try to memorize every word; just remember the sequence: *(1) Title → (2) The Blindfolded Teacher Problem → (3) Our 3 Pillars → (4) The 4 Key Numbers → (5) Conclusion.*
- **4. Remember the 4 Key Numbers:**
  - **83.88%** test accuracy on Fashion-MNIST with SmallCNN.
  - **1.03×** verification ratio (meaning 1.5 µs verification cost whether model is 64 B or 4 MiB).
  - **12 / 14** adversarial attacks detected with 0 false alarms.
  - **366** automated test cases passing with **99%** code coverage.
- **5. Files to Keep Ready on Screen:**
  - `Stage1_Report_BC-FL-SRV.docx` (your formal report)
  - `Logbook_12Weeks_BC-FL-SRV.docx` (your 12-week logbook)
  - `BC-FL-SRV_Presentation_CheatSheet_and_Defense_Guide.pdf` (this cheat-sheet guide)

Prepared for: Stage-I Major Project Viva Defense | SIES Graduate School of Technology (Autonomous)

Ayush Lanke • Bhumi Beloshe • Sakshi Bhandare